

Лабораторная работа

Простой чат на FastAPI + HTMX WebSocket · RabbitMQ · PostgreSQL

FastAPI · HTMX ws · Docker · Nginx · RabbitMQ · SQLAlchemy async · pytest · GitHub Actions

ФИО:

Хуснутдинова Венера

Группа:

426

Дата:

20.04.2026

Содержание

1. Часть 1: простой чат (вариант 0)
2. 1. Подготовка окружения
3. 2. Деплой на VPS — 2.11 Нагрузка (room.html)
4. Часть 2: БД и брокер
5. 3. Backend (db, mq, main, worker)
6. 4. Шаблоны и стили
7. 5. Проверка по шагам
8. 6. Health и тесты
9. 8. CI (GitHub Actions)
10. Чек-лист, Репозиторий, Выводы

Лабораторная работа: простой чат на FastAPI + HTMX ws extension

Часть 1: разделы 1–2 (вариант 0 и деплой). **Часть 2:** разделы 3–6 и 8 (PostgreSQL, RabbitMQ, шаблоны, проверки, тесты, CI). Подразделы 3.1, 5.2 и т.п. относятся к родительскому разделу.

Часть 1: Простой чат на сокетах

Этот вариант используется как быстрый старт:

- ▶ только FastAPI + HTMX ws extension ;
- ▶ без RabbitMQ ;
- ▶ без PostgreSQL ;
- ▶ сообщения хранятся в памяти процесса (in-memory).

Цель варианта 0

Понять базовый realtime-поток:

1. браузер подключается к /ws/{room_id} ;
2. форма отправляется через ws-send ;
3. сервер принимает сообщение и рассылает его в комнату;
4. сообщение появляется во всех открытых вкладках комнаты.

Ограничения варианта 0

- ▶ после перезапуска сервера история исчезает;
- ▶ нет гарантированной доставки;
- ▶ нет масштабирования между несколькими инстансами.

Упрощенная структура (вариант 0)

socket-mq/
app/

```
main.py
ws.py
templates/
  base.html
  room.html
static/
  styles.css
requirements.txt
Dockerfile
docker-compose.yml
```

1. Подготовка окружения

1.1 .env.example

```
APP_HOST=0.0.0.0
APP_PORT=8000
```

APP_PORT совпадает с портом, на котором **внутри контейнера** запускается Uvicorn; снаружи в docker-compose.yml проброс 8100:8000 даёт доступ с хоста по http://localhost:8100 .

1.2 requirements.txt

```
fastapi
uvicorn[standard]
jinja2
```

1.3 Dockerfile

```
FROM python:3.13-slim

WORKDIR /code
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
```

1.4 docker-compose.yml

```
services:
  web:
    build: .
    container_name: socket_mq_web_v0
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
    env_file:
      - .env
    volumes:
      - ./:/code
    ports:
      - "8100:8000"
```

1.5 Подготовьте структуру папок

Перед созданием файлов создайте каталоги проекта.

Linux/macOS:

```
mkdir -p app/templates app/static
```

Windows PowerShell:

```
mkdir app, app\templates, app\static
```

1.6 Пошагово: создаем файлы простого чата (текущий рабочий вариант)

Правило: после **каждого** шага ниже выполняйте для него блок «Проверка после шага» (запуск контейнера и/или `curl` /браузер). Не переходите к следующему шагу, пока проверка не подтверждает ожидаемое поведение или пока описано иное (например, на шаге 1 полного запуска ещё нет).

Шаг 1. Создайте `app/ws.py` (менеджер WS комнат)

```
from collections import defaultdict
from fastapi import WebSocket

class WSManager:
    def __init__(self) -> None:
        # room_id -> набор активных WebSocket-подключений.
        self.rooms: dict[str, set[WebSocket]] = defaultdict(set)
```

```
async def connect(self, room_id: str, ws: WebSocket) -> None:
    # При подключении клиент должен быть принят и добавлен в комнату.
    await ws.accept()
    self.rooms[room_id].add(ws)

def disconnect(self, room_id: str, ws: WebSocket) -> None:
    # Удаляем клиента и чистим пустую комнату.
    if room_id in self.rooms and ws in self.rooms[room_id]:
        self.rooms[room_id].remove(ws)
        if not self.rooms[room_id]:
            del self.rooms[room_id]

async def broadcast(self, room_id: str, html_fragment: str) -> None:
    # Рассылка HTML-фрагмента всем участникам комнаты.
    dead: list[WebSocket] = []
    for ws in self.rooms.get(room_id, set()):
        try:
            await ws.send_text(html_fragment)
        except Exception:
            dead.append(ws)
    for ws in dead:
        self.disconnect(room_id, ws)
```

```
manager = WSManager()
```

Проверка после шага 1. Само по себе приложение ещё не собирается: нет `main.py` и точки входа `Uvicorn`. Ошибок в файле быть не должно; переходите к шагу 2.

Шаг 2. Создайте `app/main.py` (HTTP + WS + in-memory история)

```
import json
from html import escape
from datetime import datetime
from typing import TypedDict
```

```
from fastapi import FastAPI, Request, WebSocket
from fastapi.responses import HTMLResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates

from app.ws import manager

class ChatMessage(TypedDict):
    username: str
    text: str
    created_at: str

app = FastAPI(title="Socket MQ Chat V0")
app.mount("/static", StaticFiles(directory="app/static"), name="static")
templates = Jinja2Templates(directory="app/templates")
# История в памяти процесса: после рестарта сервера очищается.
messages_by_room: dict[str, list[ChatMessage]] = {}

def render_message_fragment(msg: ChatMessage) -> str:
    # Формируем HTML для HTMX out-of-band вставки в блок #messages.
    return (
        '<div id="messages" hx-swap-oob="beforeend">'
        '<article class="msg">'
        "<header>"
        f"<strong>{escape(msg['username'])}</strong>"
        f"<small>{escape(msg['created_at'])}</small>"
        "</header>"
        f"<p>{escape(msg['text'])}</p>"
        "</article>"
        "</div>"
    )

def parse_ws_payload(raw: str) -> ChatMessage | None:
    # ws-send присылает JSON; превращаем его в валидное сообщение.
```

```
try:
    data = json.loads(raw)
except json.JSONDecodeError:
    return None

username = str(data.get("username", "")).strip()
text = str(data.get("text", "")).strip()
if not username or not text:
    return None

return {
    "username": username,
    "text": text,
    "created_at": datetime.utcnow().isoformat(),
}

@app.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse(
        request=request,
        name="room.html",
        context={"request": request, "room_id": "room-1"},
    )

@app.get("/rooms/{room_id}", response_class=HTMLResponse)
async def room_page(request: Request, room_id: str):
    return templates.TemplateResponse(
        request=request,
        name="room.html",
        context={"request": request, "room_id": room_id},
    )

@app.websocket("/ws/{room_id}")
async def ws_room(ws: WebSocket, room_id: str):
    await manager.connect(room_id, ws)
    try:
```



```
# Новому клиенту сразу отправляем накопленную историю комнаты.
for msg in messages_by_room.get(room_id, []):
    await ws.send_text(render_message_fragment(msg))

while True:
    raw = await ws.receive_text()
    msg = parse_ws_payload(raw)
    if not msg:
        continue

    # Сохраняем сообщение в in-memory историю.
    messages_by_room.setdefault(room_id, []).append(msg)
    # Рассылаем сообщение всем клиентам комнаты.
    await manager.broadcast(room_id,
render_message_fragment(msg))
    finally:
        # Гарантированно удаляем соединение при любом завершении цикла.
        manager.disconnect(room_id, ws)

@app.get("/health")
async def health():
    return {"status": "ok"}
```

Проверка после шага 2. В корне проекта (где `docker-compose.yml`) скопируйте окружение и поднимите контейнер, если ещё не делали:

```
cp .env.example .env
docker compose up -d --build
curl -s http://localhost:8100/health
```

Должно вернуться `{"status":"ok"}`. Страницы `/` и `/rooms/room-1` пока отдадут ошибку шаблона — файлы `base.html` и `room.html` появятся на шагах 3–4. Это нормально.

Шаг 3. Создайте шаблон `app/templates/base.html`

```
<!doctype html>
<html lang="ru">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>Socket MQ Chat V0</title>
    <!-- Базовая библиотека HTMX -->
    <script src="https://unpkg.com/htmx.org@1.9.12"></script>
    <!-- Расширение для WebSocket-интеграции (ws-connect / ws-send) -->
    <script src="https://unpkg.com/htmx-ext-ws@2.0.2/ws.js"></script>
    <link rel="stylesheet" href="/static/styles.css" />
  </head>
  <body>
    <!-- Контент каждой страницы подставляется в этот блок -->
    <main class="container">{% block content %}{% endblock %}</main>
  </body>
</html>
```

Проверка после шага 3. Пересоберите или перезапустите **web**, чтобы подхватить шаблон:

```
docker compose up -d --build
curl -s http://localhost:8100/health
```

/health по-прежнему JSON. Страница комнаты заработает после шага 4 (room.html).

Шаг 4. Создайте шаблон app/templates/room.html

Полный листинг с кнопками «Отправить 100» / «Отправить 1000» и скриптом sendBurst — в **разд. 2.11**; скопируйте код оттуда в файл.

Проверка после шага 4.

```
docker compose up -d --build
```

Откройте в браузере `http://localhost:8100/rooms/room-1` , отправьте тестовое сообщение. Чат должен работать по WebSocket; оформление доработаете на шаге 5 (styles.css).

Шаг 5. Создайте app/static/styles.css

```
body {  
    margin: 0;  
    font-family: Arial, sans-serif;  
    background: #f5f6f8;  
}  
  
.container {  
    max-width: 760px;  
    margin: 0 auto;  
    padding: 24px;  
}  
  
.messages {  
    min-height: 320px;  
    max-height: 500px;  
    overflow-y: auto;  
    background: #fff;  
    border: 1px solid #ddd;  
    border-radius: 8px;
```

```
padding: 12px;
margin-bottom: 12px;
}

.msg {
border-bottom: 1px solid #eee;
padding: 8px 0;
}

.msg header {
display: flex;
justify-content: space-between;
}

.message-form {
display: grid;
grid-template-columns: 160px 1fr 120px;
gap: 8px;
}

.burst-controls {
display: flex;
gap: 8px;
margin-bottom: 12px;
}
```

Проверка после шага 5 (итог варианта 0).

```
docker compose up -d --build
```

- ▶ `curl -s http://localhost:8100/health → {"status":"ok"} ;`
- ▶ `http://localhost:8100/rooms/room-1` — страница со стилями, форма и кнопки загрузки (**разд. 2.11**);
- ▶ две вкладки с той же комнатой: сообщение из одной видно в другой.

2. Быстрый деплой на VPS через git + ssh

Данные сервера:

- ▶ SSH: `alekseeva@81.90.182.174`
- ▶ Домен: `alekseeva.h1n.ru`

2.1 Первый вход

```
ssh alekseeva@81.90.182.174
```

2.2 Подготовка сервера (один раз)

```
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
```

Пояснение:

- ▶ `sudo systemctl enable --now docker` - запускает Docker сейчас и включает автозапуск после перезагрузки сервера.
- ▶ `sudo usermod -aG docker $USER` - добавляет текущего пользователя в группу `docker`, чтобы запускать команды без `sudo`.

После этого переподключитесь по SSH.

2.3 Клонирование проекта

```
mkdir -p /opt/socket-mq
cd /opt/socket-mq
git clone <URL_ВАШЕГО_РЕПОЗИТОРИЯ> app
cd app
```

2.4 Настройка .env

Создайте файл `.env` рядом с `docker-compose.yml`:

```
APP_HOST=0.0.0.0
APP_PORT=8000
```

Остальные переменные для варианта 0 не требуются. Проброс портов — как в **разд. 1.4** (`8100:8000`).

2.5 Запуск приложения

```
docker compose up -d --build
```

```
docker compose ps
```

2.6 Nginx конфиг для домена

Создайте файл `/etc/nginx/sites-available/alekseeva.h1n.ru` :

```
server {
    listen 80;
    server_name alekseeva.h1n.ru;

    location / {
        proxy_pass http://127.0.0.1:8100;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /ws/ {
        proxy_pass http://127.0.0.1:8100/ws/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
    }
}
```

Активируйте конфиг:

```
sudo ln -s /etc/nginx/sites-available/alekseeva.h1n.ru /etc/nginx/sites-enabled/
```

```
sudo nginx -t  
sudo systemctl reload nginx
```

Если получили предупреждение `conflicting server name ... ignored`, значит у домена есть дублирующий конфиг (частая ситуация после ручных правок/старых сайтов).
Проверьте активные конфиги и оставьте только один для `alekseeva.h1n.ru`:

```
ls -l /etc/nginx/sites-enabled  
sudo grep -R "server_name alekseeva.h1n.ru" /etc/nginx/sites-available  
/etc/nginx/sites-enabled  
sudo rm /etc/nginx/sites-enabled/alekseeva  
sudo nginx -t  
sudo systemctl reload nginx
```

2.7 HTTPS сертификат

```
sudo certbot --nginx -d alekseeva.h1n.ru
```

2.8 Проверка

- ▶ `http://alekseeva.h1n.ru/health`
- ▶ `https://alekseeva.h1n.ru/health`
- ▶ локально на сервере backend отвечает: `curl -i http://127.0.0.1:8100/health`

2.9 Обновление деплоя (каждый раз)


```
ssh alekseeva@81.90.182.174
cd /opt/socket-mq/app
git pull
docker compose up -d --build
```

2.10 Принудительное обновление (сотрет локальные правки tracked-файлов)

Сначала определите актуальную ветку на сервере:

```
git branch -r
```

Затем выполните принудительное выравнивание (пример для `origin/master`):

```
git fetch origin
git reset --hard origin/master # замени на нужную ветку
git clean -fd
```

2.11 Имитация нагрузки: шаблон `room.html` и кнопки «100» / «1000»

Шаг 4. Создайте `app/templates/room.html`

В этом файле задаются форма с `ws-send`, блок «Отправить 100» / «Отправить 1000» и скрипт `sendBurst`: для каждого номера меняется поле `text`, вызывается `form.requestSubmit()`, расширение `WebSocket` для HTMX шлёт JSON по уже открытому соединению. В **части 1** сообщения остаются в памяти **web**; в **части 2** — цепочка `RabbitMQ` → `worker` → `PostgreSQL` → рассылка по `WebSocket`; кнопки удобны для демонстрации очередей и задержек. Тот же шаблон для части 2 — в **разд. 4.2**; стили (включая `.burst-controls`) — **разд. 1.6 шаг 5** и **разд. 4.3**.

```
{% extends "base.html" %}
{% block content %}
<h1>Комната: {{ room_id }}</h1>

<section id="chat-root" hx-ext="ws" ws-connect="/ws/{{ room_id }}">
  <div id="messages" class="messages"></div>

  <form id="message-form" class="message-form" ws-send>
    <input type="text" name="username" placeholder="Ваш ник" required />
    <input type="text" name="text" placeholder="Сообщение" required />
    <button type="submit">Отправить</button>
  </form>

  <div class="burst-controls">
    <button type="button" id="send-100"
onclick="sendBurst(100)">Отправить 100</button>
    <button type="button" id="send-1000"
onclick="sendBurst(1000)">Отправить 1000</button>
  </div>

  <script>
    let burstBusy = false;

    function setFormDefaultsIfEmpty() {
      const form = document.getElementById('message-form');
      const usernameInput = form.querySelector('input[name="username"]');
      const textInput = form.querySelector('input[name="text"]');

      if (!usernameInput.value.trim()) usernameInput.value = 'load-test';
      if (!textInput.value.trim()) textInput.value = 'Нагрузочное
сообщение';
    }

    async function sendBurst(count) {
      if (burstBusy) return;
      burstBusy = true;

      const form = document.getElementById('message-form');
      const usernameInput = form.querySelector('input[name="username"]');
```

```
const textInput = form.querySelector('input[name="text"]');

setFormDefaultsIfEmpty();
const baseText = textInput.value.trim();
usernameInput.focus();

const btn100 = document.getElementById('send-100');
const btn1000 = document.getElementById('send-1000');
if (btn100) btn100.disabled = true;
if (btn1000) btn1000.disabled = true;

let i = 1;
const batchSize = 20;

while (i <= count) {
  const end = Math.min(count, i + batchSize - 1);
  for (; i <= end; i++) {
    textInput.value = `${baseText} #${i}`;
    form.requestSubmit();
  }
  await new Promise((r) => setTimeout(r, 0));
}

burstBusy = false;
if (btn100) btn100.disabled = false;
if (btn1000) btn1000.disabled = false;
}
</script>
</section>
{% endblock %}
```

Сценарий проверки нагрузки:

1. Открыть одну или несколько вкладок с комнатой `room-1` :
 - ▶ локально: `http://localhost:8100/rooms/room-1` ;
 - ▶ на VPS: `http://alekseeva.h1n.ru/rooms/room-1` .
1. Ввести ник (если не введён, код подставит `load-test` автоматически).
2. Нажать «Отправить 100» или «Отправить 1000» и наблюдать:
 - ▶ как быстро появляются сообщения;
 - ▶ как ведёт себя браузер и сервер при серии из сотен сообщений;
 - ▶ как это отличается при одном экземпляре сервера и при нескольких экземплярах (в варианте с брокером сообщений и базой данных, часть 2).

Часть 2: Чат с базой данных и брокером сообщений

Клиент как в части 1 (**HTMX**, `ws-send` , `WebSocket`). Отличие — на сервере: **web** не пишет в БД напрямую и не шлёт HTML сразу всем; сообщение идёт через **RabbitMQ** → **worker** (запись в **PostgreSQL**) → снова **RabbitMQ** → фоновая задача **web** `consume_persisted_events` и **broadcast** в комнату.

По шагам: (1) JSON с формы попадает в **web**, публикация в **RabbitMQ** с ключом `chat.message.created` . (2) **worker** читает очередь, вставляет строку в `messages` , публикует `chat.message.persisted` . (3) **web** в `consume_persisted_events` забирает «persisted», собирает HTML и вызывает `manager.broadcast` .

Ниже — листинги для варианта с `app/` , `worker.py` в корне, шаблонами и стилями; можно развивать проект из части 1 или собрать каталог заново. Нужны `docker-compose.yml` (сервисы `web` , `worker` , `postgres` , `rabbitmq`) и `.env` по **разд. 3.0**.

3. Backend (приложение и фоновые процессы)

Файлы **разд. 3.1–разд. 3.5** вводите по порядку; **промежуточный запуск** всего стека после одного только `db.py` или только `mq.py` не требуется — приложение и `worker` должны быть описаны целиком. После того как добавлены **разд. 3**, **разд. 4**, актуальный `docker-compose.yml` и `.env` (часть 2), переходите к **разд. 5** и проверяйте запуск **по каждому подпункту 5.1, 5.2, ...** с тестовым обращением к приложению.

3.0 Файл переменных окружения `.env` (образец для копирования в свой `.env`)

`APP_HOST=0.0.0.0`

```
APP_PORT=8000
```

```
DATABASE_URL=postgresql+asyncpg://chat:chat@postgres:5432/chat
```

```
RABBITMQ_URL=amqp://guest:guest@rabbitmq:5672/
```

```
MQ_EXCHANGE=chat.events
```

```
MQ_QUEUE_INCOMING=chat.messages.incoming
```

```
MQ_QUEUE_PERSISTED=chat.messages.persisted
```

```
MQ_ROUTING_KEY_CREATED=chat.message.created
```

```
MQ_ROUTING_KEY_PERSISTED=chat.message.persisted
```

Проброс **web** 8100:8000 — как в **разд. 1.4**; проверки ниже — <http://127.0.0.1:8100> .

3.1 app/db.py

```
from datetime import datetime
import os
from sqlalchemy import DateTime, Integer, String, Text
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
from sqlalchemy.ext.asyncio import create_async_engine,
async_sessionmaker, AsyncSession
```

```
DATABASE_URL = os.getenv("DATABASE_URL",
"postgresql+asyncpg://chat:chat@postgres:5432/chat")
```

```
class Base(DeclarativeBase):
    pass
```

```
class Message(Base):
    __tablename__ = "messages"

    id: Mapped[int] = mapped_column(Integer, primary_key=True,
```

```
autoincrement=True)
    room_id: Mapped[str] = mapped_column(String(100), index=True)
    username: Mapped[str] = mapped_column(String(64))
    text: Mapped[str] = mapped_column(Text)
    created_at: Mapped[datetime] = mapped_column(DateTime,
default=datetime.utcnow, index=True)

engine = create_async_engine(DATABASE_URL, future=True, echo=False)
SessionLocal = async_sessionmaker(bind=engine, class_=AsyncSession,
expire_on_commit=False)

async def init_models() -> None:
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
```

3.2 app/ws.py

```
from collections import defaultdict
from fastapi import WebSocket

class WSManager:
    def __init__(self) -> None:
        self.rooms: dict[str, set[WebSocket]] = defaultdict(set)

    async def connect(self, room_id: str, ws: WebSocket) -> None:
        await ws.accept()
        self.rooms[room_id].add(ws)

    def disconnect(self, room_id: str, ws: WebSocket) -> None:
        if room_id in self.rooms and ws in self.rooms[room_id]:
            self.rooms[room_id].remove(ws)
```

```
        if not self.rooms[room_id]:
            del self.rooms[room_id]

    async def broadcast(self, room_id: str, payload: str) -> None:
        dead = []
        for ws in self.rooms.get(room_id, set()):
            try:
                await ws.send_text(payload)
            except Exception:
                dead.append(ws)
        for ws in dead:
            self.disconnect(room_id, ws)
```

```
manager = WSManager()
```

3.3 app/mq.py

```
import json
import os
import asyncio
import aio_pika

RABBITMQ_URL = os.getenv("RABBITMQ_URL",
    "amqp://guest:guest@rabbitmq:5672/")
MQ_EXCHANGE = os.getenv("MQ_EXCHANGE", "chat.events")
MQ_QUEUE_INCOMING = os.getenv("MQ_QUEUE_INCOMING",
    "chat.messages.incoming")
MQ_QUEUE_PERSISTED = os.getenv("MQ_QUEUE_PERSISTED",
    "chat.messages.persisted")
MQ_ROUTING_KEY_CREATED = os.getenv("MQ_ROUTING_KEY_CREATED",
    "chat.message.created")
MQ_ROUTING_KEY_PERSISTED = os.getenv("MQ_ROUTING_KEY_PERSISTED",
```

```
"chat.message.persisted")
```

```
class MQ:
    def __init__(self):
        self.connection = None
        self.channel = None
        self.exchange = None

    async def connect(self):
        last_error = None
        for _ in range(20):
            try:
                self.connection = await
aio_pika.connect_robust(RABBITMQ_URL)
                break
            except Exception as exc:
                last_error = exc
                await asyncio.sleep(1)
        if self.connection is None:
            raise RuntimeError(f"Cannot connect to RabbitMQ:
{last_error}")
        self.channel = await self.connection.channel()
        self.exchange = await self.channel.declare_exchange(
            MQ_EXCHANGE, aio_pika.ExchangeType.TOPIC, durable=True
        )
        await self.channel.declare_queue(MQ_QUEUE_INCOMING, durable=True)
        await self.channel.declare_queue(MQ_QUEUE_PERSISTED,
durable=True)
        q_in = await self.channel.get_queue(MQ_QUEUE_INCOMING)
        q_out = await self.channel.get_queue(MQ_QUEUE_PERSISTED)
        await q_in.bind(self.exchange,
routing_key=MQ_ROUTING_KEY_CREATED)
        await q_out.bind(self.exchange,
routing_key=MQ_ROUTING_KEY_PERSISTED)

    async def publish(self, routing_key: str, payload: dict):
        body = json.dumps(payload).encode("utf-8")
        msg = aio_pika.Message(body=body,
```



```
delivery_mode=aio_pika.DeliveryMode.PERSISTENT)
    await self.exchange.publish(msg, routing_key=routing_key)

    async def close(self):
        if self.connection:
            await self.connection.close()

mq = MQ()
```

3.4 app/main.py

```
import asyncio
import json
from html import escape
from datetime import datetime
from contextlib import asynccontextmanager
from fastapi import FastAPI, Form, HTTPException, Request, WebSocket,
WebSocketDisconnect
from fastapi.responses import HTMLResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates

from app.db import init_models
from app.mq import mq, MQ_QUEUE_PERSISTED, MQ_ROUTING_KEY_CREATED
from app.ws import manager

templates = Jinja2Templates(directory="app/templates")
persisted_task: asyncio.Task | None = None

async def consume_persisted_events():
    queue = await mq.channel.get_queue(MQ_QUEUE_PERSISTED)
```

```
async with queue.iterator() as iterator:
    async for message in iterator:
        async with message.process(requeue=True):
            data = json.loads(message.body.decode("utf-8"))
            html_fragment = (
                '<div id="messages" hx-swap-oob="beforeend">'
                '<article class="msg">'
                '<header>'
                f"<strong>{escape(str(data['username']))}</strong>"
                f"<small>{escape(str(data['created_at']))}</small>"
                "</header>"
                f"<p>{escape(str(data['text']))}</p>"
                "</article>"
                "</div>"
            )
            await manager.broadcast(
                data["room_id"],
                html_fragment,
            )
```

```
@asynccontextmanager
```

```
async def lifespan(app: FastAPI):
    global persisted_task
    await init_models()
    await mq.connect()
    persisted_task = asyncio.create_task(consume_persisted_events())
    yield
    if persisted_task:
        persisted_task.cancel()
    await mq.close()
```

```
app = FastAPI(title="Socket MQ Chat", lifespan=lifespan)
app.mount("/static", StaticFiles(directory="app/static"), name="static")
```

```
@app.get("/", response_class=HTMLResponse)
```

```
async def home(request: Request):
```

```
    return templates.TemplateResponse(
        request=request,
        name="room.html",
        context={"request": request, "room_id": "room-1"},
    )

@app.get("/rooms/{room_id}", response_class=HTMLResponse)
async def room_page(request: Request, room_id: str):
    return templates.TemplateResponse(
        request=request,
        name="room.html",
        context={"request": request, "room_id": room_id},
    )

@app.post("/rooms/{room_id}/messages")
async def send_message(room_id: str, username: str = Form(...), text: str = Form(...)):
    username = username.strip()
    text = text.strip()
    if not username or not text:
        raise HTTPException(status_code=400, detail="username/text cannot be empty")

    payload = {
        "room_id": room_id,
        "username": username,
        "text": text,
        "created_at": datetime.utcnow().isoformat(),
    }
    await mq.publish(MQ_ROUTING_KEY_CREATED, payload)
    return {"status": "queued"}

@app.websocket("/ws/{room_id}")
async def ws_room(ws: WebSocket, room_id: str):
    await manager.connect(room_id, ws)
    try:
```

```
while True:
    raw = await ws.receive_text()
    try:
        data = json.loads(raw)
    except json.JSONDecodeError:
        continue

    username = str(data.get("username", "")).strip()
    text = str(data.get("text", "")).strip()
    if not username or not text:
        continue

    payload = {
        "room_id": room_id,
        "username": username,
        "text": text,
        "created_at": datetime.utcnow().isoformat(),
    }
    await mq.publish(MQ_ROUTING_KEY_CREATED, payload)
except WebSocketDisconnect:
    manager.disconnect(room_id, ws)
except Exception:
    manager.disconnect(room_id, ws)


@app.get("/health")
async def health():
    return {"status": "ok"}
```

3.5 worker.py

```
import asyncio
import json
from sqlalchemy import insert
```

```
import aio_pika

from app.db import Message, SessionLocal, init_models
from app.mq import (
    RABBITMQ_URL,
    MQ_EXCHANGE,
    MQ_QUEUE_INCOMING,
    MQ_ROUTING_KEY_CREATED,
    MQ_ROUTING_KEY_PERSISTED,
)

async def run_worker():
    await init_models()
    connection = await aio_pika.connect_robust(RABBITMQ_URL)
    channel = await connection.channel()
    exchange = await channel.declare_exchange(
        MQ_EXCHANGE, aio_pika.ExchangeType.TOPIC, durable=True
    )
    queue = await channel.declare_queue(MQ_QUEUE_INCOMING, durable=True)
    await queue.bind(exchange, routing_key=MQ_ROUTING_KEY_CREATED)

    async with queue.iterator() as iterator:
        async for incoming in iterator:
            async with incoming.process(requeue=True):
                payload = json.loads(incoming.body.decode("utf-8"))

                async with SessionLocal() as session:
                    stmt = insert(Message).values(
                        room_id=payload["room_id"],
                        username=payload["username"],
                        text=payload["text"],
                    ).returning(Message.id, Message.created_at)
                    row = (await session.execute(stmt)).one()
                    await session.commit()

                persisted = {
                    "id": row.id,
                    "room_id": payload["room_id"],
```

```

        "username": payload["username"],
        "text": payload["text"],
        "created_at": row.created_at.isoformat(),
    }
    msg = aio_pika.Message(
        body=json.dumps(persisted).encode("utf-8"),
        delivery_mode=aio_pika.DeliveryMode.PERSISTENT,
    )
    await exchange.publish(msg,
routing_key=MQ_ROUTING_KEY_PERSISTED)

if __name__ == "__main__":
    asyncio.run(run_worker())

```

4. Шаблоны и стили

Те же идеи, что в части 1 (HTMX, ws-connect , ws-send); заголовок в base.html — **Socket MQ Chat** (в варианте 0 часто **V0**). Кнопки загрузки и **sendBurst** — см. **разд. 2.11**; сравнение in-memory и цепочки с брокером/БД.

4.1 app/templates/base.html

```

<!doctype html>
<html lang="ru">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1"
  />
  <title>Socket MQ Chat</title>
  <script src="https://unpkg.com/htmx.org@1.9.12"></script>
  <script src="https://unpkg.com/htmx-ext-ws@2.0.2/ws.js"></script>
  <link rel="stylesheet" href="/static/styles.css" />

```

```

</head>
<body>
  <main class="container">{% block content %}{% endblock %}</main>
</body>
</html>

```

4.2 app/templates/room.html

Тот же шаблон, что в **разд. 2.11** (форма `ws-send`, кнопки «Отправить 100» / «Отправить 1000», `sendBurst`). Ниже — полный текст для части 2.

```

{% extends "base.html" %}
{% block content %}
<h1>Комната: {{ room_id }}</h1>

<section id="chat-root" hx-ext="ws" ws-connect="/ws/{{ room_id }}">
  <div id="messages" class="messages"></div>

  <form id="message-form" class="message-form" ws-send>
    <input type="text" name="username" placeholder="Ваш ник" required />
    <input type="text" name="text" placeholder="Сообщение" required />
    <button type="submit">Отправить</button>
  </form>

  <div class="burst-controls">
    <button type="button" id="send-100"
onclick="sendBurst(100)">Отправить 100</button>
    <button type="button" id="send-1000"
onclick="sendBurst(1000)">Отправить 1000</button>
  </div>

  <script>
    let burstBusy = false;

    function setFormDefaultsIfEmpty() {

```

```
const form = document.getElementById('message-form');
const usernameInput = form.querySelector('input[name="username"]');
const textInput = form.querySelector('input[name="text"]');

if (!usernameInput.value.trim()) usernameInput.value = 'load-test';
if (!textInput.value.trim()) textInput.value = 'Нагрузочное
сообщение';
}

async function sendBurst(count) {
  if (burstBusy) return;
  burstBusy = true;

  const form = document.getElementById('message-form');
  const usernameInput = form.querySelector('input[name="username"]');
  const textInput = form.querySelector('input[name="text"]');

  setFormDefaultsIfEmpty();
  const baseText = textInput.value.trim();
  usernameInput.focus();

  const btn100 = document.getElementById('send-100');
  const btn1000 = document.getElementById('send-1000');
  if (btn100) btn100.disabled = true;
  if (btn1000) btn1000.disabled = true;

  let i = 1;
  const batchSize = 20;

  while (i <= count) {
    const end = Math.min(count, i + batchSize - 1);
    for (; i <= end; i++) {
      textInput.value = `${baseText} #${i}`;
      form.requestSubmit();
    }
    await new Promise((r) => setTimeout(r, 0));
  }

  burstBusy = false;
}
```



```
        if (btn100) btn100.disabled = false;
        if (btn1000) btn1000.disabled = false;
    }
</script>
</section>
{% endblock %}
```

4.3 app/static/styles.css

```
body {
    margin: 0;
    font-family: Arial, sans-serif;
    background: #f5f6f8;
}

.container {
    max-width: 760px;
    margin: 0 auto;
    padding: 24px;
}

.messages {
    min-height: 320px;
    max-height: 500px;
    overflow-y: auto;
    background: #fff;
    border: 1px solid #ddd;
    border-radius: 8px;
    padding: 12px;
    margin-bottom: 12px;
}

.msg {
    border-bottom: 1px solid #eee;
```

```
padding: 8px 0;
}

.msg header {
display: flex;
justify-content: space-between;
}

.message-form {
display: grid;
grid-template-columns: 160px 1fr 120px;
gap: 8px;
}

.burst-controls {
display: flex;
gap: 8px;
margin-bottom: 12px;
}
```

Перед разд. 5: убедитесь, что в проекте есть все листинги **разд. 3–разд. 4**, `worker.py`, `docker-compose.yml` части 2 и `.env` по **разд. 3.0**.

5. Проверка по шагам

Команды — из **корня проекта** (рядом `docker-compose.yml` и `.env`).

Правило: после **каждого** из **разд. 5.1–разд. 5.5** выполните указанный там **запуск** и **проверку** (браузер, `curl`, логи, при необходимости БД). К следующему подпункту переходите только если результат совпадает с ожидаемым.

5.1 Инфраструктура

Команда:

```
docker compose up -d postgres rabbitmq
```

Проверка (приложение **web** на этом шаге ещё не запускаем):

- ▶ `docker compose ps` — контейнеры **postgres** и **rabbitmq** в состоянии `running` ;
- ▶ RabbitMQ UI открывается (порт `15672` в `docker-compose.yml`).

В `docker-compose.yml` для PostgreSQL задан проброс `5433:5432` : с вашего компьютера к базе можно подключиться на `localhost:5433` (логин/пароль из `environment` сервиса `postgres`), а приложение внутри Docker по-прежнему использует в `DATABASE_URL` хост **postgres** и порт **5432** внутри сети контейнеров. Так удаётся избежать конфликта, если на хосте уже занят стандартный порт **5432**.

5.2 Сервис web

Команда:

```
docker compose up -d --build web
```

Проверка (тестовый запуск приложения):

```
curl -s http://127.0.0.1:8100/health
```

- ▶ ответ `{"status":"ok"}` ;
- ▶ в браузере открывается `http://127.0.0.1:8100/rooms/room-1` .

5.3 Worker

Команда:

```
docker compose up -d --build worker
```

Проверка:

```
docker compose logs worker --tail 50
```

- ▶ процесс worker не перезапускается в цикле;
- ▶ в логах нет ошибки `relation "messages" does not exist`.

5.4 Realtime (две вкладки)

Проверка:

1. Откройте две вкладки `http://127.0.0.1:8100/rooms/room-1`.
2. Отправьте сообщение из одной вкладки.
3. Сообщение должно появиться в обеих вкладках.
4. В DevTools → Network → WS должен быть запрос к `/ws/room-1`.
5. Если сообщения не появляются, проверьте:
 - ▶ `docker compose logs web --tail 100` ;
 - ▶ `docker compose logs worker --tail 100` ;
 - ▶ `docker compose exec -T rabbitmq rabbitmqctl list_queues name messages consumers` .

5.5 База данных

Проверка:

- ▶ в таблице `messages` появляются записи после отправки (через `psql` или клиент к БД);
- ▶ история из PostgreSQL в шаблон не подгружается: в интерфейсе — только новые сообщения после сохранения worker и рассылки `persisted` .

5.6 Деплой на VPS (часть 2 вместо варианта 0 из разд. 2)

разд. 2 — один контейнер **web** и короткий `.env` . Для части 2:

- ▶ `.env` — полный набор по **разд. 3.0** (PostgreSQL, RabbitMQ, очереди, ключи маршрутизации); значения должны совпадать с `docker-compose.yml` (или согласованно изменены везде).

- ▶ **Запуск:** после `git pull` — `docker compose up -d --build` в каталоге с compose части 2; `docker compose ps` — подняты **web, worker, postgres, rabbitmq**.
- ▶ **Nginx:** как в **разд. 2.6**, `proxy_pass` на `127.0.0.1:8100` при пробросе `8100:8000` менять не нужно. После правок конфига: `sudo nginx -t`, `sudo systemctl reload nginx`.
- ▶ **Ресурсы и порты:** нагрузка выше, чем у варианта 0. Порты **5433, 5672, 15672** на хосте закрыть от интернета; для пользователей — **80/443**.
- ▶ **Обновления:** обычно `docker compose up -d --build` после `git pull`; без пересборки образов — только если менялись не файлы приложения.
- ▶ **Проверка:** `/health` и комната по домену — как в **разд. 5.2–5.4**, но с вашим хостом.

6. Проверка health и тесты

Вручную:

```
curl -s http://127.0.0.1:8100/health
```

pytest.ini в корне (нужен для импорта `app` при `pytest` и в CI):

```
[pytest]
pythonpath = .
testpaths = tests
```

tests/test_health.py — проверка **GET /health** через `TestClient`; **monkeypatch** отключает реальные `init_models`, `mq.connect` и фоновую `consume_persisted_events`, чтобы не требовать PostgreSQL/RabbitMQ (в **Starlette 1.x** у `TestClient` нет `lifespan="off"`).

```
import asyncio

import pytest
from fastapi.testclient import TestClient

import app.main as main

def test_health(monkeypatch: pytest.MonkeyPatch) -> None:
    async def noop() -> None:
        return None

    async def fake_consume() -> None:
        await asyncio.Event().wait()

    monkeypatch.setattr(main, "init_models", noop)
    monkeypatch.setattr(main.mq, "connect", noop)
    monkeypatch.setattr(main, "consume_persisted_events", fake_consume)

    with TestClient(main.app) as client:
        response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "ok"}
```

tests/ws_check.py — не pytest: скрипт `python tests/ws_check.py` при поднятом Compose. Два WebSocket на `room-1`, затем POST на `/rooms/room-1/messages`; в консоль выводятся ответы обоих сокетов (цепочка HTTP → очередь → worker → БД → очередь → WS). Пакет **websockets** — из `requirements.txt`. В **разд. 8** в CI выполняется только `pytest`.

```
docker compose run --rm web pytest -q
```

```
python tests/ws_check.py
```

(второй запуск — с хоста, когда Compose поднят и порт **8100** доступен.)

8. Этап CI (GitHub Actions)

На `push` в `master` — установка зависимостей и `pytest -q` (см. **разд. 6**), чтобы ловить поломки до деплоя.

Добавьте `.github/workflows/ci.yml` (после первого push проверьте вкладку **Actions**).

8.1 Пример `.github/workflows/ci.yml`

```
name: CI

on:
  push:
    branches: [master]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - run: pip install -r requirements.txt
      - run: pytest -q
```

`python-version` должен совпадать с версией в **Dockerfile** / локальной разработке (в проекте — **3.13**).

8.2 Что сдать (дополнительно к отчёту)

- ▶ файл `.github/workflows/ci.yml` в репозитории;
- ▶ скриншот успешного запуска workflow (**Actions** → выбранный run → зелёная галочка).

8.3 По желанию

- ▶ `docker build` в job;
- ▶ линтер (**ruff** / **flake8**) перед `pytest` ;
- ▶ триггер `pull_request` на `master` — добавьте в `on:` при необходимости.

Чек-лист

Этап	Статус
Часть 1 — вариант 0 (Docker, WS, шаблоны)	✓
Разд. 2 — деплой VPS, Nginx, HTTPS	✓
Разд. 2.11 — нагрузка 100/1000	✓
Часть 2 — PostgreSQL, RabbitMQ, worker	✓
Разд. 5 — проверки 5.1–5.6	✓
Разд. 6 — <code>pytest</code> , <code>ws_check</code>	✓
Разд. 8 — CI workflow	✓

Ссылка на репозиторий

Ссылка на репозиторий Git с полным кодом варианта с PostgreSQL и RabbitMQ:

<https://github.com/Venera-h/Chat-with-Fast-API>

Выводы

Что изучили:

В ходе лабораторной работы изучили построение realtime-чата на основе FastAPI и HTMX WebSocket. Освоили работу с брокером сообщений RabbitMQ

Сложности:

Основная сложность возникла при деплое на VPS — на сервере был предустановлен Kubernetes (k3s) (делала сначала последующую лабу), который перехватывал порт 80 и не давал Nginx обрабатывать запросы. После отключения k3s и сброса iptables потребовалась перезагрузка